

PROGRAMMING FOR PROBLEM SOLVING

UNIT 1 C Fundamentals & Decision Making

PREMANAND S

Assistant Professor

Department of Computational Intelligence

SRM Institute of Science and Technology

Kattankulathur Campus

First Year Engineering

PROGRAMMING FOR PROBLEM SOLVING

Nested if & switch Statements

*Making Decisions Within Decisions
and Handling Multiple Choices*

In this session, we will learn

- Nested if statements
- Nested if-else
- switch statement
- case, break and default
- Fall-through in switch
- if-else vs. switch

Think → Decide → Choose → Execute

Quick Recap: Decision Making in C

What have we learned so far?

In C, decision-making statements allow a program to **choose what to do** based on a condition.

`if`
`if (condition)`
Execute when
condition is true

`if-else`
`if (condition)`
 `else`
Choose between
two alternatives

`else-if Ladder`
 `if`
 `else if`
 `else`
Choose among
multiple conditions

But what if one decision depends on another decision?

That's where Nested `if` comes in!

Why Do We Need Nested if?

Think About This Situation

A student wants to attend an examination.

Question 1: Is the student's attendance $\geq 75\%$?



If YES: Check another condition

Question 2: Are the required internal marks obtained?

The Key Idea

One decision leads to another decision.

Attendance OK $\longrightarrow$ Check Internal Marks $\longrightarrow$ Decide Eligibility

This is the basic idea behind a Nested if.

Nested if: The Basic Idea

Definition

A **Nested** if is an if statement placed **inside another if or else block**.

Outer if



Inner if

General Syntax

```
if (condition1)
{
    if (condition2)
    {
        statement;
    }
}
```

Nested if: A Simple Example +ve and even

```
int n;  
scanf("%d", &n);  
if (n > 0)  
{  
    if (n % 2 == 0)  
    {  
        printf("Positive Even");  
    }  
}
```

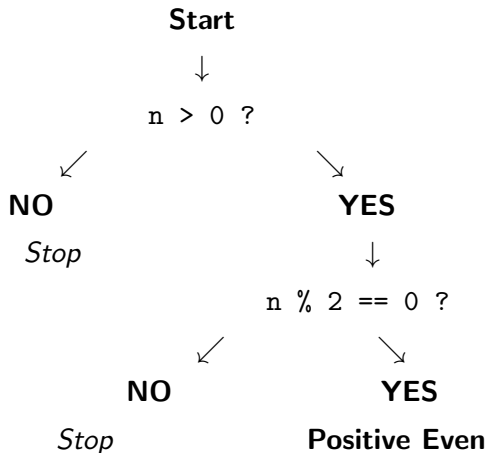
How does it work?

- 1 First check: Is the number **positive**?
- 2 If YES, check: Is the number **even**?
- 3 If both are true → Positive Even

Nested if: How Does It Work?

Execution Flow

The program evaluates the conditions **from outside to inside**.



positive and even, positive and odd or not positive

```
if (n > 0)
{
    if (n % 2 == 0)
        printf("Positive Even");
    else
        printf("Positive Odd");
}
else
    printf("Not Positive");
```

Observe the Structure

- The **outer** if checks whether the number is positive.
- The **inner** if-else checks whether it is even or odd.
- If the outer condition is false, the inner if is skipped.

Nested if: Let's Trace It

Consider the following logic

```
if (n > 0)
    ↓
if (n % 2 == 0)
```

Input	$n > 0$	$n \% 2 == 0$	Result
8	TRUE	TRUE	Positive Even
7	TRUE	FALSE	Positive Odd
-8	FALSE	–	Not Positive
0	FALSE	–	Not Positive

Important Observation

When the **outer condition** is **FALSE**,
the **inner condition** is never checked.

Outer decision first → Inner decision next

Nested if vs. else-if

They may look similar — but the purpose is different

Nested if: One decision leads to another decision

else-if: Choose one condition from multiple alternatives

Nested if

```
if (age >= 18)
{
    if (citizen == 1)
        printf("Eligible");
}
```

Dependent decisions

else-if Ladder

```
if (mark >= 90)
    ...
else if (mark >= 80)
    ...
else
    ...
```

Alternative choices

Easy Way to Remember

Nested if → “If this happens, check something else.”

Nested if: Real-World Example

Problem

A student can attend the examination only if:

- 1 Attendance is at least 75%, and
- 2 Internal marks are at least 40.

```
if (attendance >= 75)
{
    if (internal >= 40)
        printf("Eligible for Exam");
    else
        printf("Not Eligible: Low Internal Marks");
}
else
    printf("Not Eligible: Low Attendance");
```

Think About the Logic

Attendance first → Internal Marks next → Final decision

Nested if: Another Example

Problem

Write a C program to check whether a person is eligible for a driving licence.

Condition 1: Age $\geq$ 18?



If YES: Check whether the person passed the test.



Both conditions satisfied?

Nested if: Another Example

Logic

```
if (age >= 18)
{
    if (testPassed == 1)
    printf("Eligible for Licence");
}
```

Key Idea

The second decision is meaningful only after the first decision is satisfied.

Nested if: Common Mistake

Which else belongs to which if?

In nested conditions, an else is matched with the **nearest unmatched** if.

Example

```
if (a > 0)
    if (b > 0)
        printf("Both Positive");
    else
        printf("b is not positive");
```

Important

The else belongs to the **inner** if.

Use **braces** { } to make the intended structure clear.

Predict the Output



Question

What will be the **output** for each of the following values of **n**?

C Code

```
1  if (n > 0)
2  {
3      if (n % 2 == 0)
4          printf("A");
5      else
6          printf("B");
7  }
8  else
    printf("C");
```

Value of n	Output
8	?
7	?
-4	?
0	?

Answer



What will it print?



Challenge

Trace the **outer condition** first, then the **inner condition**.

Think
Step by Step
You Can Do It! 😊

Multiple Levels of Nested IF



Can We Nest More Than One **if**?

Yes! An **if** can contain another **if**, which can contain yet another **if**.

Yes,
we can
nest more
than one **if**!

General Structure

if (condition1)



if (condition2)



if (condition3)



Execute
Statement

Example: Three-Level Decision

```
1  if (age >= 18)
2  {
3      if (citizen == 1)
4      {
5          if (registered == 1)
6              printf("Can Vote");
7          }
8      }
9  }
```

Each condition
is checked only if
the previous one
is true!



Remember

More nesting = more levels of decision-making.

Use proper indentation and braces to keep the logic clear.

Clean Code
is Easy to Read
and Easy to Debug! 😊

When does Nested IF become difficult?



Imagine Many Choices...

Suppose we need to check the value of a single variable against **many fixed values**.

C Code

```
1  if (choice == 1)
2      printf("Option 1");
3  else if (choice == 2)
4      printf("Option 2");
5  else if (choice == 3)
6      printf("Option 3");
7  else if (choice == 4)
8      printf("Option 4");
9  else
10     printf("Invalid Option");
11
```



The Problem

Too many alternatives can make the code:

- Long
- Repetitive
- Harder to read and maintain



Is There a Better Way?

What if we simply want to compare **one expression with several fixed values**?



➤ Meet the **switch** statement!

What if we have 5... 10... or even more options?



Key Takeaway

Nested **if** works well for a few conditions, but for many fixed choices, the **switch statement** makes the code **simpler and cleaner**.

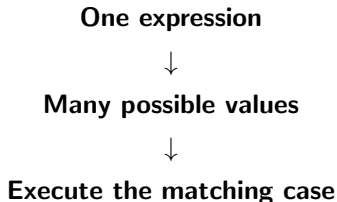
Choose the right tool for the right problem!



Introducing the switch Statement

What is switch?

When we want to choose one option from **multiple fixed choices**.



Think of a Menu

1 → Addition	2 → Subtraction
3 → Multiplication	4 → Division

Key Idea

`switch` is especially useful for menu-driven programs and fixed choices.

Basic Syntax of SWITCH



General Form

The **switch** statement is used to select one option from **multiple fixed choices** based on the value of an expression.

SWITCH is cleaner and easier to read when we have many fixed choices! 😊

C Code

```
1  switch (expression)
2  {
3      case value1:
4          statement;
5          break;
6      case value2:
7          statement;
8          break;
9      default:
10         statement;
11 }
```

Important Parts

- 1 switch**
Expression to evaluate (usually an integer or character)
- 2 case**
A possible value to match
- 3 break**
Exits from the switch statement after executing the statements
- 4 default**
Executed when no case matches (optional)

Flow of Execution



Key Takeaway

The **switch** statement is especially useful for **menu-driven programs** and **fixed choices**.

Same value.
Multiple options.
Simple and clear! ✨

First Example of switch

Problem

Write a C program to display the day of the week based on a number from 1 to 7.

```
int day;
scanf("%d", &day);
switch (day)
{
    case 1: printf("Monday"); break;
    case 2: printf("Tuesday"); break;
    case 3: printf("Wednesday"); break;
    case 4: printf("Thursday"); break;
    case 5: printf("Friday"); break;
    case 6: printf("Saturday"); break;
    case 7: printf("Sunday"); break;
    default: printf("Invalid Day");
}
```

Basic Syntax of SWITCH



General Form

The **switch** statement is used to select one option from **multiple fixed choices** based on the value of an expression.

SWITCH is cleaner and easier to read when we have many fixed choices! 😊

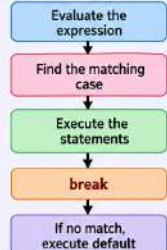
C Code

```
1  switch (expression)
2  {
3      case value1:
4          statement;
5          break;
6      case value2:
7          statement;
8          break;
9      default:
10         statement;
11 }
```

Important Parts

- 1 switch**
Expression to evaluate (usually an integer or character)
- 2 case**
A possible value to match
- 3 break**
Exits from the switch statement after executing the statements
- 4 default**
Executed when no case matches (optional)

Flow of Execution



Key Takeaway

The **switch** statement is especially useful for **menu-driven programs** and **fixed choices**.

Same value.
Multiple options.
Simple and clear! ✨

Understanding case and break



How does switch choose?

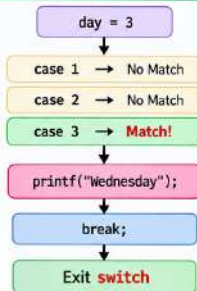
The value of the **switch** expression is compared with the values written in each **case**.

The expression is checked from top to bottom until a match is found! 😊

C Program

```
1  #include <stdio.h>
2  int main()
3  {
4      int day;
5      printf("Enter a number (1-7): ");
6      scanf("%d", &day);
7
8      switch (day)
9      {
10         case 1: printf("Monday"); break;
11         case 2: printf("Tuesday"); break;
12         case 3: printf("Wednesday"); break;
13         case 4: printf("Thursday"); break;
14         case 5: printf("Friday"); break;
15         case 6: printf("Saturday"); break;
16         case 7: printf("Sunday"); break;
17         default: printf("Invalid Day");
18     }
19     return 0;
20 }
```

Execution Flow (Example: day = 3)



case

- Represents a possible value of the expression.
- When a match is found, the statements under that **case** are executed.
- Each **case** value must be unique.

break

- Stops execution of the **switch** statement.
- Without **break**, the program will continue executing the next cases (fall-through).



Key Takeaway

The **switch** expression is compared with each **case**.

The **break** statement exits the **switch** after executing the matching case.

case selects the option, break stops the flow! 🚀

What happens without break?

21

What Happens Without **break**?

Oops! It keeps going...



Let's Remove **break**

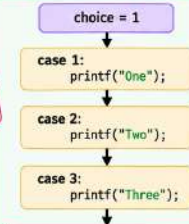
What happens when a matching **case** does not contain a **break** statement?

C Code

```
1 #include <stdio.h>
2 int main()
3 {
4     int choice;
5     scanf("%d", &choice);
6     switch (choice)
7     {
8         case 1:
9             printf("One");
10        case 2:
11            printf("Two");
12        case 3:
13            printf("Three");
14    }
15    return 0;
16 }
```

No break statements!

Execution Flow (If choice = 1)



Output: **OneTwoThree**

Once it finds a match, it doesn't stop unless we use **break**!

One

Two

Three

Output Example

Input : 1
Output: **OneTwoThree**



Even though choice is 1, all the statements from **case 1**, **case 2** and **case 3** are executed.



Fall-Through

Without **break**, execution **continues into the following cases**, even when they do not match.

Always use **break** (unless you intentionally want fall-through)!

The default case



What if no case matches?

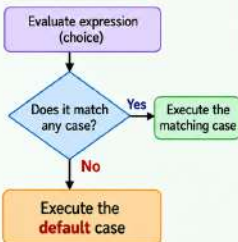
The **default case** is executed when the value of the **switch** expression does not match any **case**.

default is like the else of a switch! 😊

C Code

```
1  #include <stdio.h>
2  int main()
3  {
4      int choice;
5      printf("Enter your choice: ");
6      scanf("%d", &choice);
7
8      switch (choice)
9      {
10         case 1:
11             printf("Addition\n");
12             break;
13         case 2:
14             printf("Subtraction\n");
15             break;
16         default:
17             printf("Invalid Choice\n");
18     }
19     return 0;
20 }
```

How It Works?



Example Execution

Input : 1
Output: **Addition**

Input : 5
Output: **Invalid Choice**

Key Points

- **default** is executed when no **case** matches.
- It is **optional**, but recommended.
- Usually placed at the end.
- Helps handle **invalid** or **unexpected inputs**.



Remember

The **default** case is **optional**, but it is good practice to handle unexpected or invalid values.

Good programs also handle wrong inputs! 💡

Using switch with characters

23

Using **switch** with Characters

Not just numbers!
Characters work too!



Can switch work with characters?

Yes! A character can also be used as the **switch** expression.

Same **switch**...
Different
type of choices!

C Code

```
1 #include <stdio.h>
2 int main()
3 {
4     char grade;
5     printf("Enter grade (A/B/C): ");
6     scanf(" %c", &grade);
7
8     switch (grade)
9     {
10         case 'A':
11             printf("Excellent");
12             break;
13         case 'B':
14             printf("Very Good");
15             break;
16         case 'C':
17             printf("Good");
18             break;
19         default:
20             printf("Invalid Grade");
21     }
22     return 0;
23 }
```

How It Works?



Example Execution

Input: A	Matches case 'A'
Output: Excellent	
Input: B	Matches case 'B'
Output: Very Good	
Input: C	Matches case 'C'
Output: Good	
Input: D	No match → default
Output: Invalid Grade	

Key Points

- Characters are written in **single quotes** ('A', 'B', 'C').
- Each case value must be **unique**.
- **default** is optional, but recommended.
- Useful for menu options, grades, and choices.



Important!

Character constants in case are written using **single quotes**: 'A', 'B', 'C'.

Switch works
with integers, characters
(and even more!)

Using switch with characters



Can switch work with characters?

Yes! A character can also be used as the **switch** expression.

Same **switch**...
Different
type of choices!



C Code

```
1 #include <stdio.h>
2 int main()
3 {
4     char grade;
5     printf("Enter grade (A/B/C): ");
6     scanf(" %c", &grade);
7
8     switch (grade)
9     {
10         case 'A':
11             printf("Excellent");
12             break;
13         case 'B':
14             printf("Very Good");
15             break;
16         case 'C':
17             printf("Good");
18             break;
19         default:
20             printf("Invalid Grade");
21     }
22     return 0;
23 }
```

How It Works?



Example Execution

Input: A Output: Excellent	Matches case 'A'
Input: B Output: Very Good	Matches case 'B'
Input: C Output: Good	Matches case 'C'
Input: D Output: Invalid Grade	No match → default

Key Points

- Characters are written in **single quotes** ('A', 'B', 'C').
- Each case value must be **unique**.
- **default** is optional, but recommended.
- Useful for menu options, grades, and choices.



Important!

Character constants in case are written using **single quotes**: 'A', 'B', 'C'.

Switch works
with integers, characters
(and even more!)

switch vs if-else-if



When should we use each one?

Both can handle multiple choices, but they are useful in **different situations**.

if-else-if

- Works with **complex conditions**
- Supports **relational operators** (>, <, >=, <=, ==, !=)
- Supports **logical operators** (&&, ||, !)
- Useful for **ranges** (e.g., marks, age, temperature)

```
if (mark >= 90)
    printf("Grade A");
else if (mark >= 75)
    printf("Grade B");
else if (mark >= 50)
    printf("Grade C");
else
    printf("Grade D");
```

Good for
ranges and
complex
conditions!



switch

- Best for **fixed values**
- Compares one expression with multiple constant values
- Code is often **cleaner** and easier to read
- Useful for menu-driven programs, simple choices

```
switch (choice)
{
    case 1: printf("Addition"); break;
    case 2: printf("Subtraction"); break;
    case 3: printf("Multiplication"); break;
    default: printf("Invalid Choice");
}
```

Good for
fixed choices!
Simple and
neat!



Situation	Better Choice	Example
Fixed choices (one value among many)	switch	Menu option (1, 2, 3, 4)
Ranges	if-else-if	Marks (≥ 90), Age (< 18)
Complex conditions	if-else-if	age ≥ 18 && citizen == 1



Easy Rule!

Fixed values → **switch**

Complex conditions / ranges → **if-else**

Choose the
right tool for
the right problem!



What can we use in switch?



The switch Expression

In C, the expression used with switch should produce an **integral value**.

Commonly Used

- **int** (integer values)
- **char** (character values)
- **enum** (enumeration constants)

These
produce
integral
values!

Examples:

```
int choice;  
switch (choice) { ... }  
  
char grade;  
switch (grade) { ... }
```

Important Rules

- 1 Each case value must be **unique**.
- 2 A case value must be a **constant** (literal or constant expression).
- 3 case values **cannot be variables**.



Remember!

The case labels are checked at compile time, so they must be constant values.

Valid case values

```
case 1: // integer constant  
case 2: // integer constant  
case 'A': // character constant  
case 100: // constant expression (e.g., macro)
```



These are
valid!

Not valid case values

```
case x: // x is a variable  
case n+1: // expression with variable  
case 3.14: // not an integer constant  
case "A": // string (not allowed)
```



These are
not valid!



Key Takeaway

One expression + fixed constant cases = **switch**

Keep it simple
Keep it correct! 😊

Example: Menu-Driven Calculator

Problem

Write a C program that accepts two numbers and allows the user to choose an arithmetic operation.

```
int a, b, choice;
scanf("%d %d", &a, &b);
printf("1.Add 2.Subtract 3.Multiply 4.Divide");
scanf("%d", &choice);
switch (choice)
{
    case 1: printf("%d", a+b); break;
    case 2: printf("%d", a-b); break;
    case 3: printf("%d", a*b); break;
    case 4: printf("%d", a/b); break;
    default: printf("Invalid Choice");
}
```

Multiple cases for one action



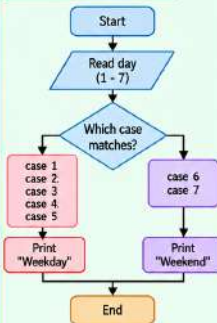
Can Multiple Cases Execute the Same Code?

Yes! Multiple case labels can be grouped when they should perform the same action.

C Program

```
1 #include <stdio.h>
2 int main()
3 {
4     int day;
5     printf("Enter day (1-7): ");
6     scanf("%d", &day);
7
8     switch (day)
9     {
10         case 1:
11         case 2:
12         case 3:
13         case 4:
14         case 5:
15             printf("Weekday");
16             break;
17         case 6:
18         case 7:
19             printf("Weekend");
20             break;
21         default:
22             printf("Invalid Day");
23     }
24     return 0;
25 }
```

How It Works?



Example Execution

Input: 3
Output: **Weekday**

Matches case 3
(executes weekday block)

Input: 7
Output: **Weekend**

Matches case 7
(executes weekend block)

Input: 8
Output: **Invalid Day**

No match
(executes default)

Key Points

- 1 Several **case** labels can share the same block of statements.
- 2 Useful when different values mean the same thing.
- 3 Use **break** after the common statement.
- 4 Helps reduce code repetition.



Key Idea!

Several cases can share the same block of statements.

Write smart code,
not repetitive code!



Switch: Predict the Output



Question

What will be the **output** for each value of **choice**?



C Program

```
1  switch (choice)
2  {
3      case 1:
4          printf("A");
5          break;
6      case 2:
7      case 3:
8          printf("B");
9          break;
10     case 4:
11         printf("C");
12         break;
13     default:
14         printf("D");
15 }
```

Read carefully...
Trace the flow!

Which case will execute?



Fill in the Output

choice	Output
1	?
2	?
3	?
4	?
5	?



Hint:

Cases 2 and 3 share the same action!
If no case matches, **default** will execute.



Challenge!

Watch carefully: cases 2 and 3 share the same action!

Think like a programmer!



switch: Solution and Trace

Let's Check the Answers

Trace the program carefully: find the matching case, then follow the execution until break.

choice	Matching Case	Action	Output
1	case 1	Print A	A
2	case 2	Print B	B
3	case 3	Print B	B
4	case 4	Print C	C
5	default	Print D	D

Notice What Happened

- case 2 and case 3 share the same action.
- Each matching case reaches a break.
- choice = 5 matches no case, so default executes.

Common Mistakes in switch



Watch Out!

A few small mistakes can change the behavior of a **switch** statement.

Write carefully
Think logically
Debug easily! 😊

1 Missing break

```
switch (choice) {  
    case 1:  
        printf("One");  
    case 2:  
        printf("Two");  
    break;  
}
```

If **break** is missing, control falls through to the next case.

Output for choice = 1
"OneTwo"

Oops!
I forgot
break!



3 Variable as Case Value

```
int x = 2;  
switch (choice) {  
    case x:  
        printf("HL");  
    break;  
}
```

Case labels must be **constant values** (literals or constant expressions).

This will give a **compilation error!**

Can we use
a variable
here?



2 Duplicate Cases

```
switch (choice) {  
    case 1:  
        printf("A");  
    break;  
    case 1:  
        printf("B");  
    break;  
}
```

Each case value must be **unique**.

This will give a **compilation error!**

Same case
value again?
Not allowed!



4 Forgetting default

```
switch (choice) {  
    case 1:  
        printf("A");  
    break;  
    case 2:  
        printf("B");  
    break;  
    /* no default */  
}
```

If no case matches, no action will be performed.

Use **default** to handle unexpected values.

Always keep
a default
case!



Good Practice

- ✓ Use **unique** case values
- ✓ Include proper **break** statements
- ✓ Use constant expressions
- ✓ Add a meaningful **default** case

Correct code
does what you expect! 😊

Nested switch



Can We Nest a switch?

Yes! A **switch** statement can be placed inside another **switch** statement.

Same idea as nested if! 😊

Idea

Outer switch
(main choice)

case 1:

Inner switch
(sub choice)

Execute
sub-option

⚡ A switch
inside a switch!

C Program

```
1 #include <stdio.h>
2 int main()
3 {
4     int mainChoice, subChoice;
5     printf("1. Arithmetic\n2. Exit\n");
6     printf("Enter main choice: ");
7     scanf("%d", &mainChoice);
8
9     switch (mainChoice)
10    {
11        case 1:
12            printf("1. Add\n2. Subtract\n");
13            printf("Enter sub choice: ");
14            scanf("%d", &subChoice);
15            switch (subChoice)
16            {
17                case 1: printf("Add\n"); break;
18                case 2: printf("Subtract\n"); break;
19                default: printf("Invalid sub choice\n");
20            }
21            break;
22        case 2:
23            printf("Exit\n");
24            break;
25        default:
26            printf("Invalid main choice\n");
27    }
28    return 0;
29 }
```

Example Execution

Step 1: Main Menu

1. Arithmetic
2. Exit

Enter main choice: 1

Step 2: Sub Menu (inside case 1)

1. Add
2. Subtract

Enter sub choice: 2

Output:
Subtract

Try It Yourself! ✎

- Run the program with different choices.
- Observe how the inner switch works only when case 1 is selected.
- Try entering an invalid choice and see the default messages.

Nested switch
is useful for
menu-driven
programs! 😊



Key Idea

Outer switch selects the **main option**.
Inner switch selects the **sub-option**.

Break at the
right places! ⚡

Combining switch and if



Can We Use if Inside a switch?

Yes! A **case** can contain an **if-else** statement.

switch and if
can work together! 😊

C Program Example

```
1  #include <stdio.h>
2  int main()
3  {
4      int choice, a;
5      printf("Enter choice: ");
6      scanf("%d", &choice);
7      printf("Enter a value: ");
8      scanf("%d", &a);
9
10     switch (choice)
11     {
12         case 1:
13             if (a > 0)
14                 printf("Positive\n");
15             else
16                 printf("Not Positive\n");
17             break;
18         case 2:
19             printf("Another Operation\n");
20             break;
21         default:
22             printf("Invalid Choice\n");
23     }
24     return 0;
25 }
```

An if-else
inside a case!

How It Works?

switch (choice)

Choose the
main operation

if (condition)

Make an additional
decision

switch → Choose an operation → if → Make an additional decision

Example Explanation

1 Case 1:

- Checks if the value **a** is greater than 0.
- If yes → prints "**Positive**".
- If no → prints "**Not Positive**".

2 Case 2:

- Prints "Another Operation".

3 Default:

- Handles invalid choices.



Key Idea

switch handles the main choice; **if** handles an additional condition.

Use the right tool
at the right place! 😊

Ex: Combining switch and if



Problem

Create a calculator using **switch**.
For division, check whether the denominator is zero.



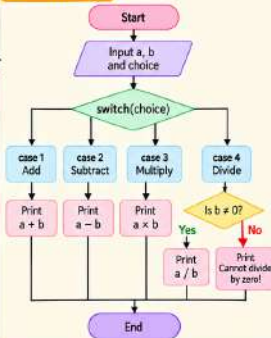
Use the right tools for the right conditions! 😊

C Program

```
1 #include <stdio.h>
2 int main()
3 {
4     int choice, a, b;
5     printf("Enter two numbers: ");
6     scanf("%d %d", &a, &b);
7
8     printf("\nMenu:\n");
9     printf("1. Add\n");
10    printf("2. Subtract\n");
11    printf("3. Multiply\n");
12    printf("4. Divide\n");
13    printf("Enter your choice: ");
14    scanf("%d", &choice);
15
16    switch (choice)
17    {
18        case 1: printf("Sum = %d\n", a + b); break;
19        case 2: printf("Difference = %d\n", a - b); break;
20        case 3: printf("Product = %d\n", a * b); break;
21        case 4:
22            if (b != 0)
23                printf("Quotient = %d\n", a / b);
24            else
25                printf("Cannot divide by zero!\n");
26            break;
27        default: printf("Invalid Choice!\n");
28    }
29    return 0;
30 }
```

switch for main choice, **if** for special condition!

Flowchart



Sample Run

Enter two numbers: 10 5
Menu:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter your choice: 4
Quotient = 2

Enter two numbers: 10 0
Menu:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter your choice: 4
Cannot divide by zero!



Key Takeaway

switch chooses the operation; **if** handles the special condition.



Real-World Learning

In real programs, we often combine control structures to handle different situations.

Think.
Check.
Code Better! 😊